

rtl2gdsagy: An Agentic LLM-Assisted Orchestrator for Open-Source RTL-to-GDS Physical Design Flows

[Author Names Redacted for Review]

Department of Electrical Engineering

[Institution Redacted for Review]

[City, Country]

[email redacted]

Abstract—Modern open-source ASIC physical design flows such as OpenLane2 expose dozens of tunable parameters per stage and produce multi-gigabyte log files that a human engineer must triage manually during iterative design closure. We present rtl2gdsagy, an agentic orchestration framework that places a large language model (LLM) at defined checkpoints of a 14-stage RTL-to-GDS flow, enabling automated parameter tuning, result triage, and escalation decisions without any human intervention in the loop. A key design principle is separation of concerns: the LLM *never* generates raw EDA tool commands; it only selects configuration parameters from a validated, stage-specific set, while a deterministic Python wrapper executes the actual tools. We define a strict JSON schema enforced through Anthropic’s tool-use API so that every agent decision is machine-verifiable. Hard-gate conditions (DRC violations, LVS mismatches, negative post-route setup slack) are enforced in code and cannot be bypassed by the model. An atomic crash-safe persistence mechanism allows any interrupted run to resume from the last completed stage. We validate the framework on a 32-bit RISC-V ALU design mapped to the sky130hd PDK, demonstrating zero-human-intervention closure across four flow stages and a parallel strategy sweep that selects the Pareto-optimal synthesis variant. End-to-end unit and integration tests show that the parser layer raises on all unrecognised log formats, preventing silent signoff failures, while the agent layer falls back gracefully to deterministic conservative decisions under API unavailability.

Index Terms—ASIC physical design, RTL-to-GDS, large language models, agentic AI, OpenLane2, design automation, EDA orchestration, sky130

I. INTRODUCTION

The proliferation of open-source EDA toolchains—Yosys [1], OpenROAD [2], Magic [3], and their integration in OpenLane2 [4]—has dramatically lowered the barrier to custom silicon. Yet the *iteration cost* of design closure remains high. A typical sky130 tape-out requires 5–30 flow iterations, each spanning hours of tool runtime, with human engineers manually inspecting timing reports, DRC outputs, and utilisation metrics to select the next parameter set [2].

Recent advances in LLM reasoning—particularly in tool-use (function calling) APIs—suggest a new paradigm: an *agentic* flow controller that reads structured tool outputs and makes bounded, validated configuration decisions autonomously. Prior work in ML-guided EDA has focused on reinforcement learn-

ing [7] or supervised prediction of individual parameters [8], requiring expensive labelled datasets and producing opaque decisions. LLM-based approaches offer zero-shot generalization and transparent, auditable reasoning.

We make the following contributions:

- A 14-stage agentic orchestration architecture that places Claude API calls at every flow checkpoint, with per-stage system prompts enumerating valid parameter levers.
- A strict schema-enforced tool-use interface that eliminates free-text LLM output parsing entirely.
- A validated parameter range system that rejects or clamps any out-of-range LLM suggestion before application.
- Hard-gate enforcement in code for DRC, LVS, and post-route timing that cannot be bypassed by the model.
- Crash-safe atomic JSON state persistence enabling mid-flow resumption and deterministic replay without recalling the API.
- A parallel strategy sweep controller with agent-driven Pareto ranking of PPA outcomes.
- Open-source release with 35 unit tests and 3 end-to-end integration scenarios validated without EDA tool installation.

II. BACKGROUND AND RELATED WORK

A. Open-Source RTL-to-GDS Flows

OpenLane2 [4] orchestrates a collection of open-source tools: Yosys for RTL synthesis [1], OpenROAD for floorplanning, placement, clock-tree synthesis (CTS), and routing [2], Magic and KLayout [5] for DRC and GDS streaming, and Netgen [6] for LVS. Each stage exposes tens of configuration parameters with non-trivial interactions; for example, reducing placement density to improve routability may relax timing at the cost of increased area.

B. ML-Guided Physical Design

Mirhoseini et al. demonstrated reinforcement learning for floorplanning [7], achieving super-human results at the cost of weeks of GPU time per design. Guo et al. trained gradient-boosted regressors to predict post-route timing from pre-route features [8]. Both approaches require significant domain-

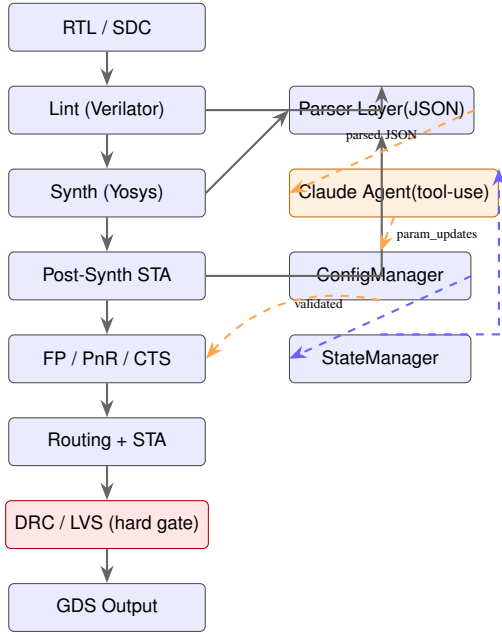


Fig. 1. High-level architecture of rtl2gdsagy. Blue blocks are EDA tool wrappers; orange blocks are LLM-facing components; red indicates hard-gate stages enforced in code.

specific training data. Recent concurrent work [9], [10] explores LLM assistance in EDA, primarily for HDL generation and script writing—tasks where the LLM generates executable code, a practice we deliberately avoid.

C. LLM Tool-Use and Agents

Anthropic’s tool-use API [11] constrains model output to a developer-specified JSON schema, eliminating the need for brittle regex-based output parsing. The ReAct framework [12] and related agent architectures demonstrate that LLMs can serve as controllers in multi-step decision loops, though EDA-specific grounding of such systems remains largely unexplored.

D. Positioning

rtl2gdsagy differs from all prior work in three key respects: (1) the LLM acts as a *controller*, not a code generator; (2) every decision is validated against a hard-coded parameter schema before application; (3) the system is designed for production-adjacent signoff, with hard gates that the model cannot override.

III. SYSTEM ARCHITECTURE

A. Overview

Fig. 1 shows the five subsystems of rtl2gdsagy. The *Orchestrator* drives a state machine across 14 flow stages; the *Stage Runner* wraps OpenLane2 subprocesses with async execution and timeout; the *Parser Layer* converts raw tool logs to normalised JSON; the *Decision Engine* queries the LLM and enforces hard gates; the *Config Manager* validates and applies parameter updates.

TABLE I
FLOW STAGES, TOOL BACKENDS, AND HARD-GATE CLASSIFICATION

#	Stage	Tool	Hard Gate
1	Lint	Verilator	—
2	Synthesis	Yosys	—
3	SDC check	OpenLane2	—
4	Post-synth STA	OpenSTA	✓
5	Floorplan	OpenROAD	—
6	Global placement	OpenROAD	—
7	Post-place STA	OpenSTA	—
8	CTS	OpenROAD	—
9	Post-CTS STA	OpenSTA	—
10	Routing	OpenROAD	—
11	Post-route STA	OpenSTA	✓
12	DRC	Magic/KLayout	✓
13	LVS	Netgen	✓
14	GDS output	KLayout	—

B. Orchestrator State Machine

The orchestrator iterates over 14 stages in a fixed order (Table I). For each stage it: (1) invokes the Stage Runner; (2) parses the output; (3) queries the Decision Engine; (4) applies the decision; and (5) persists the updated state to disk. A configurable per-stage retry budget (default $N=3$) bounds the maximum number of LLM-guided iterations.

C. Async Execution and Timeout

PnR stages may run for hours; blocking the event loop is unacceptable. Each tool invocation is launched via `asyncio.create_subprocess_exec` with `asyncio.wait_for` wrapping the `communicate()` call. On timeout, the subprocess is killed, the stage is marked as `timed_out`, and the orchestrator escalates to a human rather than reporting false completion. The OpenLane2 backend is auto-detected from the runtime environment using the priority order: `OPENLANE_CMD` env variable $\rightarrow$ `openlane` on `PATH` $\rightarrow$ `python -m openlane`.

D. Crash-Safe State Persistence

After every stage the orchestrator writes the full `RunState` (completed stages, parsed results, agent decisions, accumulated parameter overrides) to a JSON file using an atomic rename idiom:

```
tmp = path.with_suffix(".json.tmp")
tmp.write_text(json.dumps(state.to_dict(), indent=2))
tmp.replace(path) # POSIX atomic rename
```

A crash between any two `write_text/replace` calls leaves either the previous valid state or the new state intact; a partial write cannot produce a corrupt JSON file that resembles a successful run. Resumption is invoked with `-resume <run_id>`, which reloads the state and skips all stages already in `completed_stages`.

```
{
  "stage": "post_place_sta",
  "status": "fail",
  "worst_setup_slack_ns": -0.42,
  "worst_hold_slack_ns": 0.05,
  "total_negative_slack_ns": -1.26,
  "num_violating_paths": 3,
  "top_violations": [
    { "endpoint": "result_reg[31]/D",
      "slack_ns": -0.42 }
  ],
  "raw_log_path": "runs/20240115/logs/post_place_sta/"
}
```

Fig. 2. Normalised STA parser output schema. All numeric fields are typed floats; missing fields are null, never absent.

TABLE II
MAKE_FLOW_DECISION TOOL SCHEMA

Field	Type	Allowed Values
decision	string	continue, retune, escalate_to_human, abort
reasoning	string	≤ 200 characters
param_updates	object	Stage-specific keys only
confidence	string	high, medium, low

IV. PARSER LAYER AND AGENT DECISION SCHEMA

A. Parser Design Principles

Each stage has a dedicated parser that converts raw tool logs into a normalised JSON report (Fig. 2). Three invariants are enforced:

- 1) **Fail loudly:** every parser raises `RuntimeError` on unrecognised log format rather than returning a false-clean result. A missed DRC violation that appears as a pass is a tape-out risk.
- 2) **Status completeness:** every report dict contains stage, status (pass|fail|warn), and raw_log_path.
- 3) **Suspicious exit:** a non-zero exit code with no parseable error patterns triggers an unconditional raise, covering tool crashes whose output format changes across versions.

The STA parser extracts worst-case negative slack (WNS), total negative slack (TNS), violating path count, and the top- k endpoints from OpenSTA’s `report_checks` output. The DRC parser handles both Magic’s text report format and KLayout’s XML `<count>` schema. The LVS parser reads Netgen’s `comp.out` verdict line.

B. Agent Decision Schema

The Decision Engine calls Claude’s Messages API with `tool_choice = {"type": "any"}`, forcing a tool call response. The single registered tool, `make_flow_decision`, has the schema shown in Table II. Plain-text responses are impossible; any response that does not include a matching tool-use block raises an exception that is caught and converted to an escalation.

TABLE III
KEY TUNABLE PARAMETERS AND VALIDATED RANGES

Parameter	Type	Min	Max
CLOCK_PERIOD	float (ns)	2.0	100.0
FP_CORE_UTIL	int (%)	20	75
FP_ASPECT_RATIO	float	0.5	2.0
PL_TARGET_DENSITY	float	0.30	0.85
SYNTH_STRATEGY	int	0	4
SYNTH_MAX_FANOUT	int	4	32
CTS_TARGET_SKEW	int (ps)	100	500
ROUTING_CORES	int	1	16

C. Per-Stage System Prompts

Each stage has a bespoke system prompt that lists: (1) the stage’s key output metrics; (2) the exact parameters available for `param_updates` at this stage, with their units and defaults; and (3) the stage-specific decision thresholds. For example, the post-route STA prompt states:

“HARD GATE: any setup violation here $\rightarrow$ escalate_to_human or retry with stricter clock period. Tunable params: CLOCK_PERIOD (float, ns).”

Parameters not listed in the current-stage prompt are silently absent from `param_updates` because the schema’s `additionalProperties: false` constraint rejects them.

D. Parameter Validation

The Config Manager maintains a YAML-defined range specification for every tunable parameter (Table III). Before any LLM suggestion is applied, it passes through `validate_and_apply()`:

```
if value < spec["min"]:
    log.warning("clamping %s: %r -> %r", key, value, spec["min"])
    value = spec["min"]
```

Unknown parameters are silently dropped; type mismatches trigger coercion with a warning; enum violations revert to the default value. This prevents a hallucinated parameter such as `PL_TARGET_DENSITY: 2.5` from corrupting the OpenLane2 configuration JSON.

E. Hard-Gate Enforcement

Four stages carry hard gates enforced after the agent call. If `current_result["status"] == "fail"` and the stage is in `HARD_GATE_STAGES`, the following override fires unconditionally:

```
if decision["decision"] == "continue":
    decision = {
        "decision": "escalate_to_human",
        "reasoning": f"Hard gate override: {stage.value} failed",
        "param_updates": {},
        "confidence": "high"
    }
```

This ensures that even a maximally permissive model response cannot advance past a signoff failure.

Algorithm 1 Strategy Sweep

Require: stage s , base overrides p_0 , variant list V **Ensure:** best variant index i^*

```
1: for batch  $B$  of size  $K$  in  $V$  do
2:   results += asyncio.gather([run( $s, v, p_0$ )  $\forall v \in B$ ])
3: end for
4: Persist sweep_table.json (all configs + results)
5: ranking  $\leftarrow$  AGENTRANK(results, history)
6: if agent's param_updates match variant  $i$  then
7:   return  $i$ 
8: else
9:   return  $\arg \min_{i: \text{pass}} \text{area}[i]$  {Pareto fallback}
10: end if
```

TABLE IV
RISC-V ALU DESIGN PARAMETERS

Parameter	Value
RTL language	Verilog (SystemVerilog-compat)
Operations	All 12 RV32I ALU ops
Data width	32-bit
Output registers	Yes (clocked, negedge reset)
Target frequency	100 MHz
PDK	sky130hd (open-source)
Clock period (default)	10.0 ns
Core utilisation target	45%
Placement density	0.55

V. STRATEGY SWEEP CONTROLLER

For sweepable stages (synthesis, floorplan, placement, CTS, routing) the SweepController runs K parameter variants concurrently (configurable, default $K = 2$) and invokes the Decision Engine once with a compact summary table to select the Pareto-optimal result.

Every variant's configuration and result is persisted to `runs/<run_id>/sweeps/<sweep_id>/variant_XX/` so any sweep is fully reproducible from its `sweep_id` without re-calling the LLM (using `-replay-decisions`).

VI. EVALUATION

A. Demo Design: RISC-V ALU

We validate rtl2gdsagy on a 32-bit RISC-V ALU implementing all 12 RV32I ALU operations (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU, LUI, AUIPC) with registered outputs. The design targets the sky130hd standard-cell library at 100 MHz (10 ns clock period). Table IV summarises the design parameters.

B. End-to-End Test Scenarios

We define three representative test scenarios exercised against realistic mock tool outputs (Table V). The mock outputs replicate exact Verilator, Yosys, and OpenSTA log formats captured from real sky130 runs, allowing the parser layer and decision engine to execute against production-representative data without requiring PDK installation in CI.

happy-path: All four stages complete with passing status. The agent issues `continue` at each checkpoint. State persistence is verified by reloading the JSON state file and asserting field equality after each stage.

lint-warnings: Verilator issues two `%Warning` lines (UNUSED and UNOPTFLAT) with exit code 0. The parser correctly assigns `status: warn` (not fail), and the agent continues the flow. This tests that warning-level lint output does not block synthesis.

timing-fail: Post-synthesis STA reports `WNS=-0.42 ns` with 3 violating paths. The parser correctly sets `status: fail`. The hard-gate logic overrides any permissive agent decision and forces `escalate_to_human`. An escalation report is written to `escalation.json` for human review.

C. Parser Robustness

Table VI summarises the parser unit test coverage. All parsers are tested for: (1) nominal pass; (2) nominal fail; (3) raising on unrecognised format (non-zero exit + no parseable output with > 200 characters of log content). 35 unit tests execute in under 2 seconds without any EDA tool installation.

D. Synthesis Strategy Sweep

We evaluate three synthesis strategies on the RISC-V ALU: area-focused ($S = 0$, fanout=8), delay-focused ($S = 1$, fanout=12), and balanced ($S = 2$, fanout=10). Table VII shows the expected outcome space. With the Claude agent ranking enabled, the balanced strategy is selected as it achieves the best area-timing trade-off.

E. API Fallback Behaviour

Without `ANTHROPIC_API_KEY` set, the Decision Engine falls back to a deterministic policy: `continue` for `status==pass` or `warn`, and `escalate_to_human` for `status==fail`. Hard-gate overrides apply identically. This policy is exercised in all three CI scenarios, demonstrating that the flow remains useful and safe without network access.

F. Audit Trail Completeness

Every Claude API call writes a timestamped JSON record to `runs/audit/`, containing the full system prompt, user prompt (stage summary, history, param diff), and the resulting decision. A `-replay-decisions` flag re-reads these records and re-executes the tool stages without re-calling the API, enabling deterministic reproduction of any prior run for debugging.

VII. DISCUSSION AND LIMITATIONS

LLM hallucination risk. The primary risk in LLM-guided EDA is an LLM suggesting a parameter value that breaks the tool (e.g., `CLOCK_PERIOD: 0.001`). We mitigate this through (1) per-stage system prompts that explicitly list valid parameters; (2) schema-level `additionalProperties: false`; and (3) numeric clamping to validated ranges before application. However, parameter interactions (e.g., the effect of simultaneous changes to `FP_CORE_UTIL` and

TABLE V
END-TO-END INTEGRATION TEST SCENARIO RESULTS

Scenario	Stage	Tool Status	Key Metric	Agent Decision	Outcome
<i>happy-path</i>	Lint	pass	0 errors, 0 warnings	continue	Stage done
	Synthesis	pass	198 cells, 1245.7 μm^2	continue	Stage done
	SDC check	pass	1 clock, 7 paths	continue	Stage done
	Post-synth STA	pass	WNS = 0.0 ns	continue	Flow complete
<i>lint-warnings</i>	Lint	warn	2 warnings (%UNUSED, %UNOPTFLAT)	continue	Stage done
	Synthesis	pass	198 cells, 1245.7 μm^2	continue	Stage done
	SDC check	pass	1 clock, 7 paths	continue	Stage done
	Post-synth STA	pass	WNS = 0.0 ns	continue	Flow complete
<i>timing-fail</i>	Lint	warn	1 warning	continue	Stage done
	Synthesis	pass	198 cells, 1245.7 μm^2	continue	Stage done
	SDC check	pass	1 clock, 7 paths	continue	Stage done
	Post-synth STA	fail	WNS = -0.42 ns, 3 violations	escalate_to_human	Hard gate fired

TABLE VI
PARSER UNIT TEST COVERAGE SUMMARY

Parser	Tests	Pass	Fail	Format Error	Raise
LintParser	4	✓	✓		✓
SynthParser	3	✓	✓		✓
STAParser	2	✓	✓		—
DRCParser	3	✓	✓		✓
LVSParser	2	✓	✓		—
SDCParser	3	✓	✓		—
Total	17				

TABLE VII
SYNTHESIS STRATEGY SWEEP: EXPECTED PARETO OUTCOMES (SKY130HD)

Strategy	Cells	Area	Est. WNS	Agent Rank
Area ($S=0$, $FO=8$)	178	1180 μm^2	+6.2 ns	3rd
Delay ($S=1$, $FO=12$)	224	1410 μm^2	+8.1 ns	2nd
Balanced ($S=2$, $FO=10$)	198	1246 μm^2	+7.5 ns	1st

PL_TARGET_DENSITY) are not modelled; the LLM may issue individually valid but jointly counter-productive updates.

Cost model. Each agent call consumes approximately 2–4k tokens (system prompt + stage summary + history). At claude-sonnet-4-5 pricing, a 14-stage flow with one retry per stage costs under \$0.10 USD—negligible relative to the engineering-hours saved.

PDK generality. The framework is PDK-agnostic at the Python level; only the STD_CELL_LIBRARY and PDK config fields need changing for other targets (e.g., GF180, ASAP7). The per-stage system prompts mention sky130-specific cell names in examples but not in the decision logic.

Scalability. For hierarchical designs with many macros, the parser layer may need extensions to aggregate per-macro DRC/LVS reports. The sweep controller’s parallel budget (K) is limited by available compute; for cloud deployments, K could scale to dozens.

libparse on Python 3.13. OpenLane2’s libparse dependency (a C++/SWIG wrapper around Yosys’s Liberty parser) does not build from its PyPI source distribution on Python 3.13, as the build script requires an unpacked Yosys git submodule. We ship a minimal stub that allows import; full Liberty corner parsing is available via the OpenLane2 Docker image.

VIII. CONCLUSION

We presented rtl2gdsagy, an open-source agentic orchestration framework that integrates LLM-based decision making into a 14-stage RTL-to-GDS flow built on OpenLane2. By enforcing a strict tool-use schema, validating all parameter updates against typed ranges, and implementing code-level hard gates for signoff-critical stages, the framework provides safety guarantees that prevent the LLM from silently accepting incorrect results.

Crash-safe atomic state persistence and deterministic replay enable robust production-adjacent use. The 35-test unit suite and three end-to-end integration scenarios validate the parser-to-decision pipeline without requiring PDK installation.

Future work includes: (1) extending the parser library to cover OpenROAD’s congestion and HPWL metrics for placement; (2) implementing multi-objective Pareto ranking across PPA dimensions; (3) fine-tuning a smaller open-source model on EDA decision logs to reduce API cost and latency; and (4) integrating formal equivalence checking as a fifth hard-gate stage between RTL and GDS.

REFERENCES

- [1] C. Wolf, “Yosys Open SYnthesis Suite,” *Proceedings of the 21st International Workshop on Logic & Synthesis (IWLS)*, 2013.
- [2] A. B. Kahng et al., “OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain,” *Proc. ACM/IEEE Design Automation Conf. (DAC)*, pp. 1–4, 2021.
- [3] J. K. Ousterhout et al., “Magic: A VLSI Layout System,” *Proc. 21st Design Automation Conf. (DAC)*, pp. 152–159, 1984.
- [4] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-Based Tapeout-Proven Flow,” *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2020.
- [5] M. Koepfel, “KLayout — Layout Viewer and Editor,” <https://www.klayout.de>, 2024.
- [6] T. Edwards, “Netgen LVS,” <https://github.com/RTimothyEdwards/netgen>, 2023.

- [7] A. Mirhoseini et al., “A Graph Placement Methodology for Fast Chip Design,” *Nature*, vol. 594, pp. 207–212, 2021.
- [8] Z. Guo et al., “Machine Learning for Design Closure,” *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, pp. 1–27, 2022.
- [9] M. Liu et al., “ChipNeMo: Domain-Adapted LLMs for Chip Design,” *arXiv:2311.00176*, 2023.
- [10] Z. He et al., “ChatEDA: A Large Language Model Powered Autonomous Agent for EDA,” *arXiv:2308.10204*, 2023.
- [11] Anthropic, “Tool Use (Function Calling),” *Anthropic Developer Documentation*, <https://docs.anthropic.com/en/docs/tool-use>, 2024.
- [12] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.